In [2]:
```
import pandas as pd
import numpy as np
```

In [3]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

In [5]:
```
df = pd.read_csv("Files/museum_visitors.csv", index_col = 0, parse_dates = [0], usecols =
df.head()
```

Out[5]:

Date	Avila Adobe
2014-01-01	24778
2014-02-01	18976
2014-03-01	25231
2014-04-01	26989
2014-05-01	36883

In [5]:
```
type(df.index)
```

Out[5]:
```
pandas.core.indexes.datetimes.DatetimeIndex
```

`parse_dates` — **pandas**da CSV fayldagi **sana (datetime) turidagi ustunlarni avtomatik ravishda sanaga aylantirib beruvchi parametr**.

`parse_dates` nima qiladi?

Agar CSV faylda sana matn ko‘rinishida bo‘lsa, pandas uni oddiy *string* sifatida o‘qiydi:

```
"2014-01"
"2015-06"
"2016-09"
```

`parse_dates=[0]` yozilsa, **1-ustun sanaga aylantiriladi**, ya’ni:

```
2014-01-01   →  Timestamp('2014-01-01')
2015-06-01   →  Timestamp('2015-06-01')
```

In [6]:
```
df = pd.read_csv("Files/museum_visitors.csv", index_col = 0, parse_dates=[0], usecols = [0
type(df.index)
```

Out[6]:
```
pandas.core.indexes.datetimes.DatetimeIndex
```

In [7]:
```
df['month'] = [i.month for i in df.index]
df.head()
```

Out[7]:

Date	Avila Adobe	month
2014-01-01	24778	1
2014-02-01	18976	2
2014-03-01	25231	3
2014-04-01	26989	4
2014-05-01	36883	5

In [8]:
```
df['year'] = [i.year for i in df.index]
df.head()
```

Out[8]:

Date	Avila Adobe	month	year
2014-01-01	24778	1	2014
2014-02-01	18976	2	2014
2014-03-01	25231	3	2014
2014-04-01	26989	4	2014
2014-05-01	36883	5	2014

Quyida **df.head()** va unga o'xshash eng ko'p ishlatiladigan DataFrame ko'rish metodlari haqida ma'lumot berilgan.

`df.head()`

`df.head()` DataFramening **birinchi 5 qatorini** ko'rsatadi.

In [9]:
```
df.head()
```

Out[9]:

Date	Avila Adobe	month	year
2014-01-01	24778	1	2014
2014-02-01	18976	2	2014
2014-03-01	25231	3	2014
2014-04-01	26989	4	2014
2014-05-01	36883	5	2014

In [10]:
```
df.head(10)    # birinchi 10 qator
```

Out[10]:

Date	Avila Adobe	month	year
2014-01-01	24778	1	2014
2014-02-01	18976	2	2014
2014-03-01	25231	3	2014
2014-04-01	26989	4	2014
2014-05-01	36883	5	2014
2014-06-01	29487	6	2014
2014-07-01	32378	7	2014
2014-08-01	37680	8	2014
2014-09-01	28473	9	2014
2014-10-01	27995	10	2014

`df.tail()`

`df.tail()` DataFramening **oxirgi 5 qatorini** chiqaradi.

In [11]:
```
df.tail()
```

Out[11]:

Date	Avila Adobe	month	year
2018-07-01	23136	7	2018
2018-08-01	20815	8	2018
2018-09-01	21020	9	2018
2018-10-01	19280	10	2018
2018-11-01	17163	11	2018

In [12]:
```
df.tail(3)    # oxirgi 3 qator
```

Out[12]:

Date	Avila Adobe	month	year
2018-09-01	21020	9	2018
2018-10-01	19280	10	2018
2018-11-01	17163	11	2018

`df.info()`

`df.info()` DataFramening **ustunlar tipi**, **bo‘sh qiymatlar soni**, **xotira hajmi** haqida ma’lumot beradi.

In [13]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 59 entries, 2014-01-01 to 2018-11-01
Data columns (total 3 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Avila Adobe  59 non-null     int64
 1   month        59 non-null     int64
 2   year         59 non-null     int64
dtypes: int64(3)
memory usage: 1.8 KB
```

Unda ma’lumot turi (int, float, object, datetime), null qiymatlar bor-yo‘qligi ko‘rinadi.

`df.describe()` — statistik tahlil (faqat sonli ustunlar bilan ishlaydi)

`df.describe()` quyidagilarni beradi:

- count
- mean
- std
- min
- max
- 25%, 50%, 75% percentillar

In [14]:
```
df.describe()
```

Out[14]:

	Avila Adobe	month	year
count	59.000000	59.000000	59.000000
mean	24061.661017	6.406780	2015.966102
std	5948.997414	3.434704	1.413800
min	14035.000000	1.000000	2014.000000
25%	19469.500000	3.500000	2015.000000
50%	23136.000000	6.000000	2016.000000
75%	27502.000000	9.000000	2017.000000
max	41242.000000	12.000000	2018.000000

`df.sample()` — random qator ko‘rish

DataFramedan tasodifiy qator ko‘rsatadi:

In [15]:
```
df.sample()
```

Out[15]:

Date	Avila Adobe	month	year
2018-08-01	20815	8	2018

In [16]:
```
df.sample(3)    # 3 ta tasodifiy qator
```

Out[16]:

Date	Avila Adobe	month	year
2016-08-01	25987	8	2016
2016-02-01	17378	2	2016
2016-09-01	22897	9	2016

`df.shape` — o‘lchamlar

Qator va ustunlar sonini beradi:

In [17]:
```
df.shape
# (qatorlar_soni, ustunlar_soni)
```

Out[17]: (59, 3)

`df.columns` — ustun nomlari

In [18]:
```
df.columns
```

Out[18]:
```
Index(['Avila Adobe', 'month', 'year'], dtype='object')
```

`df.index` — index qiymatlari

In [19]:
```
df.index
```

Out[19]:
```
DatetimeIndex(['2014-01-01', '2014-02-01', '2014-03-01', '2014-04-01',
               '2014-05-01', '2014-06-01', '2014-07-01', '2014-08-01',
               '2014-09-01', '2014-10-01', '2014-11-01', '2014-12-01',
               '2015-01-01', '2015-02-01', '2015-03-01', '2015-04-01',
               '2015-05-01', '2015-06-01', '2015-07-01', '2015-08-01',
               '2015-09-01', '2015-10-01', '2015-11-01', '2015-12-01',
               '2016-01-01', '2016-02-01', '2016-03-01', '2016-04-01',
               '2016-05-01', '2016-06-01', '2016-07-01', '2016-08-01',
               '2016-09-01', '2016-10-01', '2016-11-01', '2016-12-01',
               '2017-01-01', '2017-02-01', '2017-03-01', '2017-04-01',
               '2017-05-01', '2017-06-01', '2017-07-01', '2017-08-01',
               '2017-09-01', '2017-10-01', '2017-11-01', '2017-12-01',
               '2018-01-01', '2018-02-01', '2018-03-01', '2018-04-01',
               '2018-05-01', '2018-06-01', '2018-07-01', '2018-08-01',
               '2018-09-01', '2018-10-01', '2018-11-01'],
              dtype='datetime64[ns]', name='Date', freq=None)
```

`df.nunique()` — noyob qiymatlar soni

Har bir ustundagi turli qiymatlar sonini beradi:

In [20]:
```
df.nunique()
```

Out[20]:
```
Avila Adobe    59
month          12
year            5
dtype: int64
```

Metod	Vazifasi
`df.head()`	Birinchi qatorlar
`df.tail()`	Oxirgi qatorlar
`df.info()`	Texnik diagnostika
`df.describe()`	Statistik tahlil
`df.sample()`	Random qatorlar
`df.shape`	O‘lchamlar
`df.columns`	Ustun nomlari
`df.index`	Index ro‘yxati
`df.nunique()`	Noyob qiymatlar soni

In [21]:
```
df['year'] = [i.year for i in df.index]
df.head()
```

Out[21]:

Date	Avila Adobe	month	year
2014-01-01	24778	1	2014
2014-02-01	18976	2	2014
2014-03-01	25231	3	2014
2014-04-01	26989	4	2014
2014-05-01	36883	5	2014

In [24]:
```
df2 = df.groupby(['month', 'year']).max()
df2.size
```

Out[24]: 59

`groupby()` va unga o'xshash metodlar pandasda **ma'lumotlarni guruhlash va agregat qilish** uchun ishlatiladi. Quyida bu metodlar va ularning ishlashi tushuntirilgan.

`groupby()` — ma'lumotlarni guruhlash

`groupby()` metodidan foydalanish orqali ma'lumotlarni ma'lum bir ustunlar bo‘yicha guruhlash mumkin. Guruhlashdan so‘ng, har bir guruh uchun agregat funksiyalarini (masalan, **max**, **sum**, **mean**) qo‘llash mumkin.

Sintaksisi:

```
df2 = df.groupby(['month', 'year']).max()
```

Bu yerda:

- `['month', 'year']` — **guruhlash** uchun ustunlar. Demak, **oy** va **yil** ustunlari bo‘yicha guruhlash amalga oshadi.
- `.max()` — har bir guruh uchun maksimal qiymatni hisoblash.

Misol:

In [25]:
```
df = pd.DataFrame({
    'month': [1, 1, 2, 2, 3, 3],
    'year': [2020, 2020, 2020, 2020, 2020, 2020],
    'value': [10, 20, 30, 40, 50, 60]
})

df2 = df.groupby(['month', 'year']).max()
print(df2)
```

```
            value
month year
1     2020     20
2     2020     40
3     2020     60
```

Guruhlashdan so‘ng, har bir yil va oy uchun eng katta qiymat (maksimal) qaytariladi.

`agg()` — turli agregat funksiyalarini qo‘llash

Agar bir nechta agregat funksiyalarini bir vaqtning o‘zida qo‘llash kerak bo‘lsa, `agg()` metodidan foydalanish mumkin.

Sintaksisi:

```
df.groupby(['month', 'year']).agg({
    'value': ['min', 'max', 'mean']
})
```

Bu yerda:

- `'min'`, `'max'`, va `'mean'` — har bir guruh uchun minimum, maksimum, va o‘rtacha qiymatni hisoblash.

Misol:

In [26]:
```
df2 = df.groupby(['month', 'year']).agg({
    'value': ['min', 'max', 'mean']
})
print(df2)
```

```
           value
             min max  mean
month year
1     2020    10  20  15.0
2     2020    30  40  35.0
3     2020    50  60  55.0
```

`size()` — guruh hajmini hisoblash

Agar har bir guruhdagi elementlar sonini bilmoqchi bo‘lsangiz, **`size()`** metodini qo‘llash mumkin.

Sintaksisi:

```
df.groupby(['month', 'year']).size()
```
Bu metod har bir guruhdagi elementlar sonini qaytaradi.

Misol:

In [27]:
```
df2 = df.groupby(['month', 'year']).size()
print(df2)
```

```
month  year
1      2020    2
2      2020    2
3      2020    2
dtype: int64
```

`sum()` — guruhlangan qiymatlar yig‘indisi

Guruhlangan ma'lumotlar bo‘yicha **yig‘indi** olish uchun **`sum()`** metodini ishlatish mumkin.

Misol:

In [28]:
```
df2 = df.groupby(['month', 'year']).sum()
print(df2)
```

```
           value
month year
1     2020     30
2     2020     70
3     2020    110
```

`transform()` — guruhlangan qiymatlarni saqlab qolish

Agar guruhlangan qiymatlarni asl DataFrameda saqlab qolish va ular bilan ishlashni istasangiz, `transform()` metodini ishlatishingiz mumkin. Bu metod guruhlashdan so‘ng barcha satrlar uchun hisoblangan qiymatlarni asl DataFrameda saqlaydi.

Misol:

In [29]:
```
df['max_value'] = df.groupby(['month', 'year'])['value'].transform('max')
print(df)
```

```
   month  year  value  max_value
0      1  2020     10         20
1      1  2020     20         20
2      2  2020     30         40
3      2  2020     40         40
4      3  2020     50         60
5      3  2020     60         60
```

`filter()` — guruhlarni filtrlash

`filter()` metodini guruhlarni ma'lum bir shartga asoslanib tanlash uchun ishlatishingiz mumkin.

Misol:

In [33]:
```
df2 = df.groupby(['month', 'year']).filter(lambda x: len(x) > 1)
print(df2)
```

```
   month  year  value  max_value
0      1  2020     10         20
1      1  2020     20         20
2      2  2020     30         40
3      2  2020     40         40
4      3  2020     50         60
5      3  2020     60         60
```

Bu yerda, **faqat 1 tadan ko‘p qiymatga ega guruhlar** tanlanadi.

Metod	Vazifasi
`groupby()`	Ma'lumotlarni guruhlash
`agg()`	Bir nechta agregat funksiyalarini qo‘llash
`size()`	Har bir guruhdagi elementlar sonini hisoblash
`sum()`	Guruhlangan qiymatlarning yig‘indisini olish
`transform()`	Guruhlangan qiymatlarni asl DataFramega saqlash
`filter()`	Guruhlarni filtrlash

In [36]:
```
df = pd.read_csv("Files/museum_visitors.csv", index_col = 0, parse_dates=[0], usecols = [0
df.head(10)
```

Out[36]:

Date	Avila Adobe
2014-01-01	24778
2014-02-01	18976
2014-03-01	25231
2014-04-01	26989
2014-05-01	36883
2014-06-01	29487
2014-07-01	32378
2014-08-01	37680
2014-09-01	28473
2014-10-01	27995

In [39]:
```
df['month'] = [i.month for i in df.index]
df['year'] = [i.year for i in df.index]
df.head()
```

Out[39]:

Date	Avila Adobe	month	year
2014-01-01	24778	1	2014
2014-02-01	18976	2	2014
2014-03-01	25231	3	2014
2014-04-01	26989	4	2014
2014-05-01	36883	5	2014

In [40]:
```
df2 = df.groupby(['month', 'year']).max()
df2.unstack(level=0)
```

Out[40]:

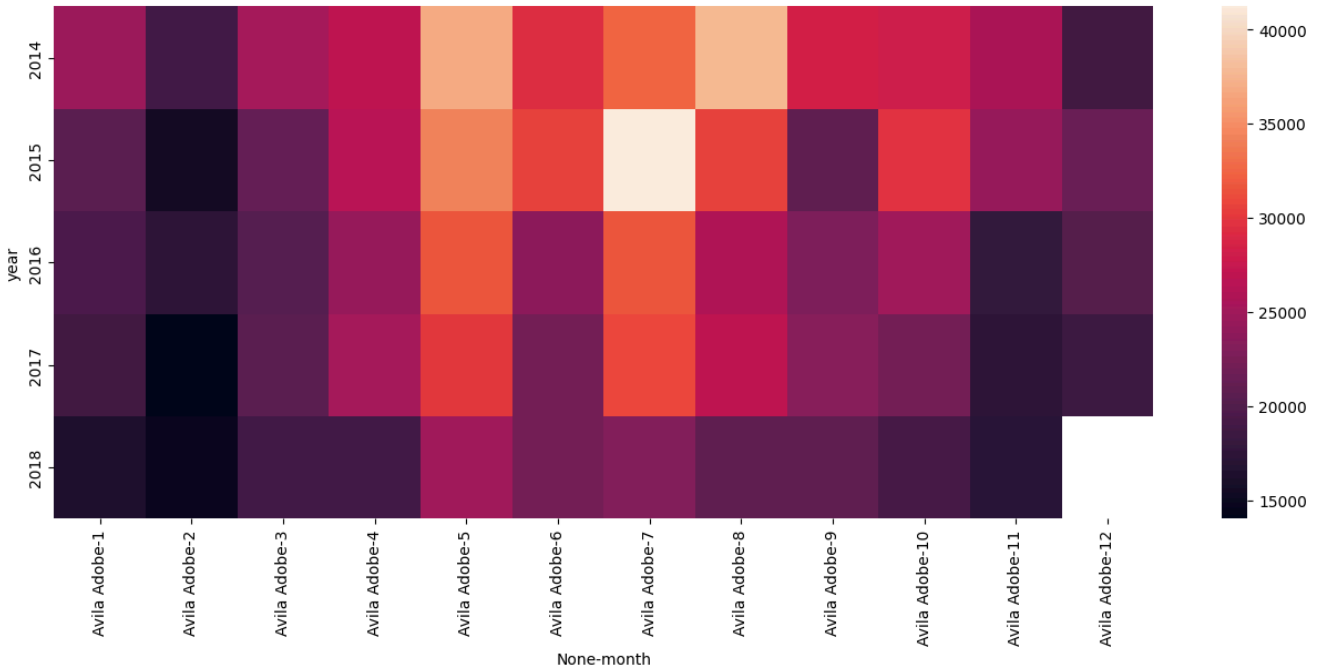
	Avila Adobe										
month	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	
year											
2014	24778.0	18976.0	25231.0	26989.0	36883.0	29487.0	32378.0	37680.0	28473.0	27995.0	25
2015	20438.0	15578.0	21297.0	26670.0	34383.0	30569.0	41242.0	30700.0	20967.0	29764.0	24
2016	19659.0	17378.0	20322.0	24521.0	31728.0	23696.0	31689.0	25987.0	22897.0	25040.0	17
2017	18792.0	14035.0	20680.0	25234.0	30029.0	22169.0	30831.0	27009.0	23403.0	22164.0	17
2018	16265.0	14718.0	19001.0	18966.0	25173.0	22171.0	23136.0	20815.0	21020.0	19280.0	17

In [41]:
```
df2.unstack(level=1)
```

Out[41]:

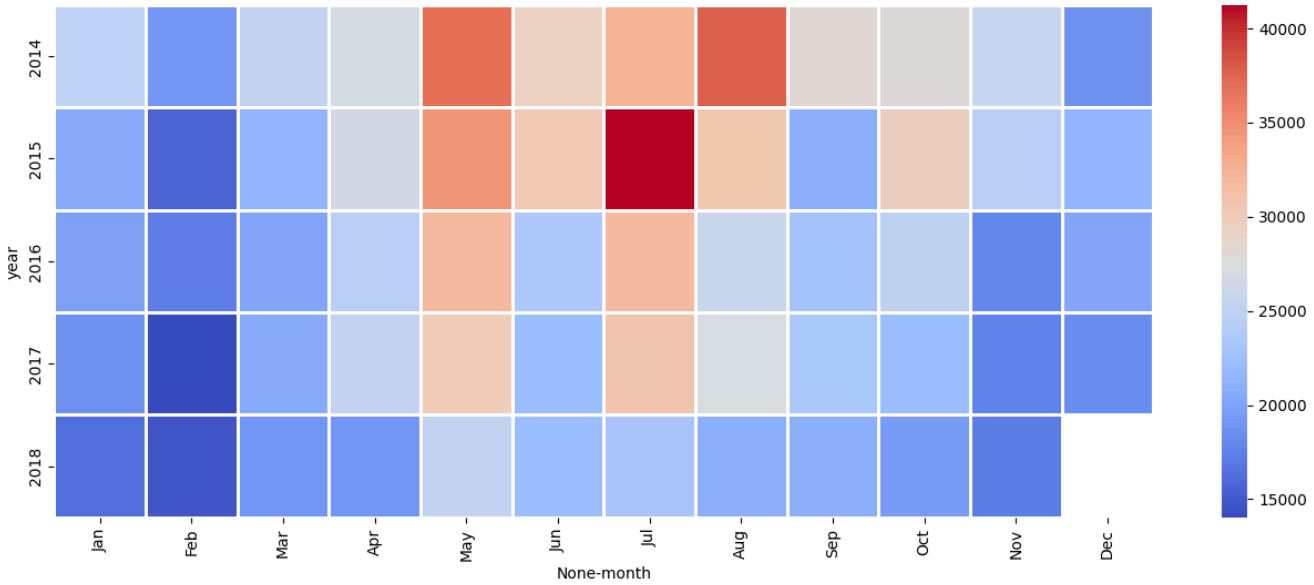
	Avila Adobe				
year	2014	2015	2016	2017	2018
month					
1	24778.0	20438.0	19659.0	18792.0	16265.0
2	18976.0	15578.0	17378.0	14035.0	14718.0
3	25231.0	21297.0	20322.0	20680.0	19001.0
4	26989.0	26670.0	24521.0	25234.0	18966.0
5	36883.0	34383.0	31728.0	30029.0	25173.0
6	29487.0	30569.0	23696.0	22169.0	22171.0
7	32378.0	41242.0	31689.0	30831.0	23136.0
8	37680.0	30700.0	25987.0	27009.0	20815.0
9	28473.0	20967.0	22897.0	23403.0	21020.0
10	27995.0	29764.0	25040.0	22164.0	19280.0
11	25691.0	24483.0	17760.0	17629.0	17163.0
12	18754.0	21426.0	20107.0	18339.0	NaN

In [46]:
```
df2 = df.groupby(['month', 'year']).max().unstack(level=0)
plt.figure(figsize=(16,6))
sns.heatmap(data=df2)
plt.show()
```



In [53]:
```
plt.figure(figsize=(16,6))
sns.heatmap(data=df2, cmap="coolwarm", linewidths=1, square=False)
xticks_labels = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
```

```
                'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
plt.xticks(np.arange(12) + .5, labels=xticks_labels)
plt.show()
```



1. `cmap` (Color Map) qiymatlari:

`cmap` parametri rang palitrasini belgilaydi, va Seaborn va Matplotlib kutubxonalari turli xil rang palitralarini taqdim etadi. Eng keng tarqalganlari:

Matplotlib-ning rang palitralari:

- **`'viridis'`**: To'liq rangli palitra (yashil, sariq, ko'k, binafsha). Odatda, yaxshilab kontrastni ta'minlaydi.
- **`'plasma'`**: Sariqdan binafsha ranggacha.
- **`'inferno'`**: Issiq ranglar palitrasi, qora, qizil va sariq tonlarda.
- **`'magma'`**: Ko'kdan pushti ranglargacha bo'lgan palitra.
- **`'cividis'`**: Ko'k va sariq ranglardan iborat kontrastli palitra.

RANGLAR PALITRASI:

- **`'YlGnBu'`**: Yellow-Green-Blue (sariq, yashil, ko'k).
- **`'coolwarm'`**: Issiqdan sovuq ranglarga o'zgarish.
- **`'Blues'`**: Barcha ranglar ko'k tonlarida.
- **`'Purples'`**: Barcha ranglar binafsha tonlarida.
- **`'RdBu'`**: Qizil va ko'k tonlarda.
- **`'PiYG'`**: Pink-to-Green (pushti-yashil).
- **`'Set1'`, `'Set2'`, `'Set3'`**: Seaborn palitralari, bir nechta ranglarni ifodalaydi.
- **`'RdYlGn'`**: Red-Yellow-Green (qizil, sariq, yashil).

boshqa palitrlar:

- **`'binary'`**: Faqat qora va oq ranglar.
- **`'Greens'`**: Yashil rang palitrasi.
- **`'Oranges'`**: To'q sariq rang palitrasi.

- `'Purples'`: Binafsha rang palitrasi.
- `'Spectral'`: Yaxshi ko'rinadigan ranglar to'plami.

To'liq rang palitrasi ro'yxatini, siz quyidagi manzilda topishingiz mumkin:

- Matplotlib color maps documentation

2. `square` qiymati:

`square` parametri faqat **True** yoki **False** qiymatlarni oladi.

- **True**: Har bir katak **kvadrat** shaklida bo'ladi.
- **False**: Har bir katak tasvirning o'lchamiga qarab shaklini o'zgartiradi.

`square` parametri **True** yoki **False** qiymatlarini oladi va heatmapning kataklari shaklini qanday ko'rsatishini belgilaydi. Quyida misollar keltirilgan.

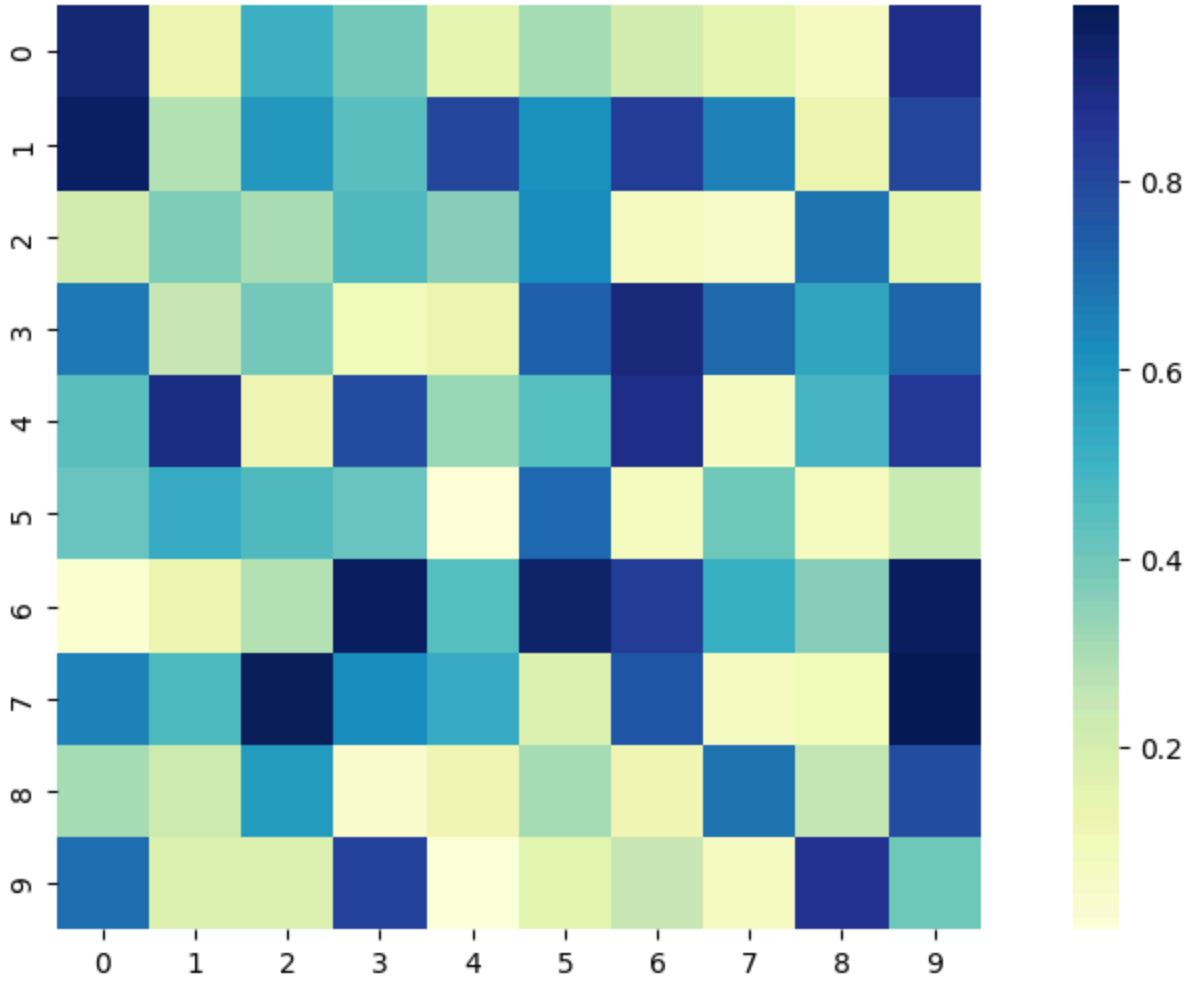
1. `square=True` (Kataklar kvadrat shaklida bo'ladi):

Agar `square=True` deb belgilasangiz, `heatmap` ning kataklari har doim kvadrat shaklida bo'ladi. Bu, tasvir o'qi bo'yicha (x va y o'qlari) teng masofada bo'lishini ta'minlaydi.

Misol:

In [59]:
```
plt.figure(figsize=(12, 6))
# Tasodifiy ma'lumotlar
data = np.random.rand(10, 10)

# Heatmap yaratish, square=True
sns.heatmap(data, square=True, cmap="YlGnBu")
plt.show()
```



- **Kataklar**: har biri **kvadrat** shaklida bo'ladi, ya'ni kenglik va balandlik teng bo'ladi.
- `square=True` shuni ta'minlaydi.

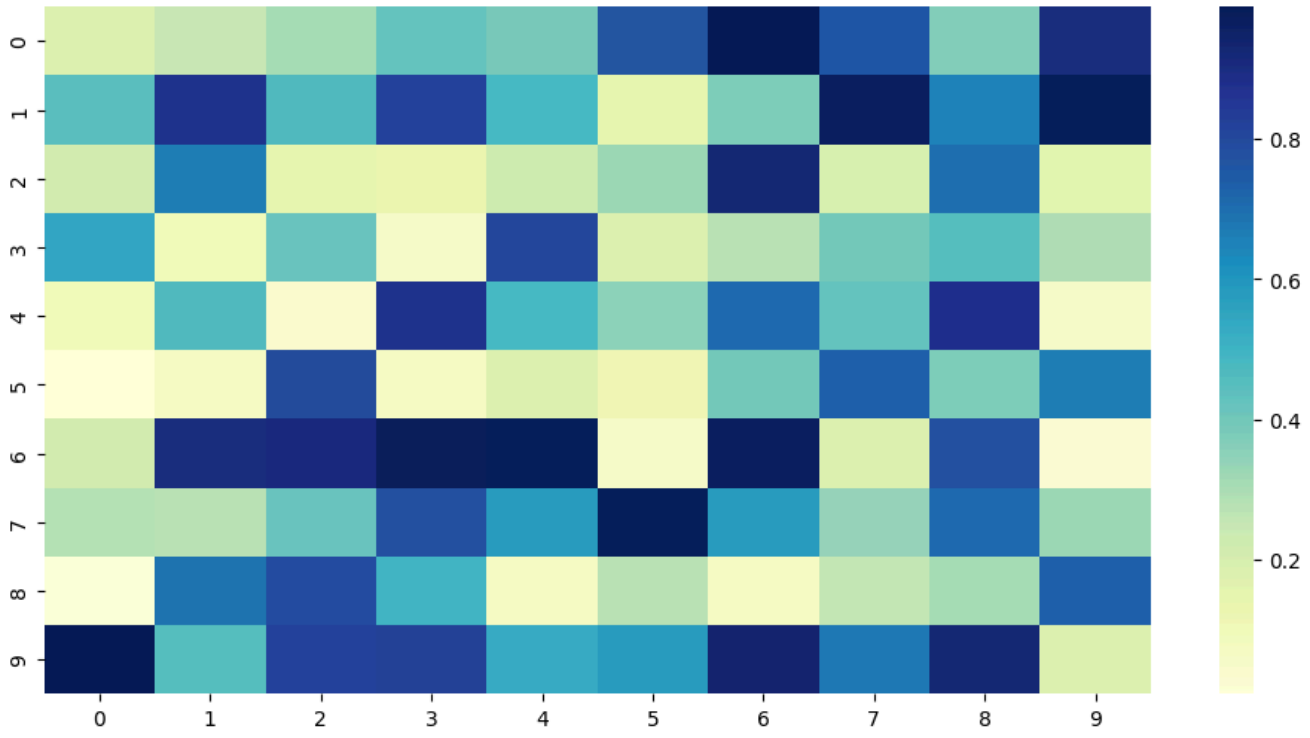
---

2. `square=False` (Kataklar to'liq o'lchamga qarab bo'ladi):

Agar `square=False` deb belgilasangiz, kataklar **kenglik** va **balandlik** (o'qlar) orasidagi nisbatga qarab o'zgaradi. Ya'ni, tasvirning hajmi o'zgarishi bilan kataklarning shakli ham o'zgaradi.

Misol:

In [57]:
```
plt.figure(figsize=(12, 6))
# Heatmap yaratish, square=False
sns.heatmap(data, square=False, cmap="YlGnBu")
plt.show()
```



Natija:

- **Kataklar**: tasvirning o'lchamiga qarab **to'rtburchak** shaklida bo'ladi. Bu, ekran o'lchamiga qarab kenglik va balandlikning farqi bo'lishi mumkin.

In [ ]: